CS543 · Web Services · GraphQL

A First Hands-On GraphQL Tutorial

Build one small app end to end — HTML/CSS → GraphQL (Node) → MySQL

Objective: Create a MySQL database of **authors and their books**, put a GraphQL API in front of it, and build a plain HTML/CSS page that reads and writes through that API.

> **Outcome:**
> A "Campus Library" page that lists authors with their books, and a form to add a book — the page talks to a GraphQL API, which talks to MySQL. One read (query) and one write (mutation), the two things every data app does.

The architecture

Three layers. The browser never touches MySQL directly — it only sends GraphQL to the API, and the API (through its resolvers) speaks SQL to the database.



Browser (front-end) → GraphQL API (Node + Apollo) → MySQL (database)

> **TASK:**
> Store authors and books in MySQL; let a web page ask "**list every author with the title and year of each of their books**" in a single request, and "**add a new book**" with a form.

Part 0 · Install the tools

Install Node.js, MySQL and VSCode in any OS (Windows / macOS / Linux).

- **Node.js** (v18 or newer) — runs the GraphQL server. Download from `nodejs.org` and install. Check it:

```
Terminal
$ node -v
v22.x.x
$ npm -v
10.x.x
```

- **MySQL** (Community Server) — the database. Install "MySQL Community Server" (and optionally MySQL Workbench for a GUI). Check it:

```
Terminal
$ mysql --version
mysql  Ver 8.x ...
```

- **VS Code** (code editor) — install the free **Live Server** extension; we use it to open the web page.

Part 1 · Create the database (MySQL)

Make a project folder anywhere, e.g. `campus-library`. Inside it, create a file `schema.sql` with the database, the two tables, and some sample rows:

```
schema.sql
-- 1. create the database
CREATE DATABASE IF NOT EXISTS gqldemo;
USE gqldemo;

-- 2. create the two tables (authors 1 --- many books)
CREATE TABLE authors (
  id      INT AUTO_INCREMENT PRIMARY KEY,
```

```
  name    VARCHAR(100) NOT NULL,
  country VARCHAR(50)
);

CREATE TABLE books (
  id        INT AUTO_INCREMENT PRIMARY KEY,
  title     VARCHAR(200) NOT NULL,
  year      INT,
  author_id INT NOT NULL,
  FOREIGN KEY (author_id) REFERENCES authors(id)
);

-- 3. put some sample rows in
INSERT INTO authors (name, country) VALUES
  ('Asha Patel', 'India'),
  ('Ravi Kumar', 'India');

INSERT INTO books (title, year, author_id) VALUES
  ('Databases 101', 2020, 1),
  ('SQL in Practice', 2022, 1),
  ('Web Services Basics', 2023, 2);
```

Run this file in MySQL. Open a terminal and log in as root, then source the file:

```
Terminal
$ mysql -u root -p
Enter password: ********
mysql> SOURCE schema.sql;
Query OK, 1 row affected ...
mysql> SELECT * FROM books;
+----+---------------------+------+-----------+
| id | title               | year | author_id |
+----+---------------------+------+-----------+
|  1 | Databases 101       | 2020 |         1 |
|  2 | SQL in Practice     | 2022 |         1 |
|  3 | Web Services Basics | 2023 |         2 |
+----+---------------------+------+-----------+
```

Part 2 · Set up the Node project

In the same `campus-library` folder, create the Node project and install the three libraries we need:

```
Terminal
$ npm init -y
$ npm install @apollo/server graphql mysql2
```

What each library is for:

- `@apollo/server` — runs the GraphQL server.
- `graphql` — the core GraphQL engine Apollo builds on.
- `mysql2` — lets Node talk to MySQL.

Part 3 · Connect Node to MySQL

Create `db.js`. This opens one shared connection pool to your database. Put your own MySQL root password in.

```
db.js
// db.js  --  one shared connection pool to MySQL
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: 'YOUR_MYSQL_PASSWORD',   // <-- change this
```

```
  database: 'gqldemo',
});

module.exports = pool;
```

Part 4 · The GraphQL API (schema + resolvers)

Create `server.js`. This one file has three parts: the **schema** (what the API offers), the **resolvers** (how each field gets its data from MySQL), and the **start** call. Read the comments — they map each piece to an idea from the lecture.

server.js

```
// server.js  --  the GraphQL API
const { ApolloServer } = require('@apollo/server');
const { startStandaloneServer } = require('@apollo/server/standalone');
const pool = require('./db');

// 1. the schema: what the API offers
const typeDefs = `#graphql
  type Author {
    id: ID!
    name: String!
    country: String
    books: [Book!]!
  }

  type Book {
    id: ID!
    title: String!
    year: Int
    author: Author!
  }

  type Query {
    authors: [Author!]!
    author(id: ID!): Author
    books: [Book!]!
  }

  type Mutation {
    addBook(title: String!, year: Int, authorId: ID!): Book!
  }
`;

// 2. the resolvers: how each field gets its data from MySQL
const resolvers = {
  Query: {
    authors: async () => {
      const [rows] = await pool.query('SELECT * FROM authors');
      return rows;
    },
    author: async (_parent, { id }) => {
      const [rows] = await pool.query('SELECT * FROM authors WHERE id = ?', [id]);
      return rows[0];
    },
    books: async () => {
      const [rows] = await pool.query('SELECT * FROM books');
      return rows;
    },
  },
  // relationship fields -- these are the ones you must write
  Author: {
    books: async (author) => {
      const [rows] = await pool.query('SELECT * FROM books WHERE author_id = ?', [author.id]);
      return rows;
    },
  },
  Book: {
    author: async (book) => {
      const [rows] = await pool.query('SELECT * FROM authors WHERE id = ?', [book.author_id]);
      return rows[0];
```

```
    },
  },
  Mutation: {
    addBook: async (_parent, { title, year, authorId }) => {
      const [result] = await pool.query(
        'INSERT INTO books (title, year, author_id) VALUES (?, ?, ?)',
        [title, year, authorId]
      );
      const [rows] = await pool.query('SELECT * FROM books WHERE id = ?', [result.insertId]);
      return rows[0];
    },
  },
};

// 3. start the server
async function start() {
  const server = new ApolloServer({ typeDefs, resolvers });
  const { url } = await startStandaloneServer(server, { listen: { port: 4000 } });
  console.log(`GraphQL API ready at ${url}`);
}
start();
```

What the resolvers are doing

- `Query.authors` / `Query.books` — run a plain `SELECT` and return the rows.
- `Author.books` — the **relationship**: given an author, fetch that author's books (the JOIN side). **This is a resolver you must write.**
- `Book.author` — the reverse link: given a book, fetch its author.
- `Mutation.addBook` — runs an `INSERT`, then returns the new row.

> **Note — fields you do NOT write**
> Notice there is no resolver for `Book.title`, `Book.year` or `Author.name`. Once a row is loaded, those fields resolve **automatically** from the row's columns. You only write resolvers for the roots and the relationships.

Part 5 · Run the API and test it

Start the server:

```
Terminal
$ node server.js
GraphQL API ready at http://localhost:4000/
```

Open `http://localhost:4000/` in your browser — Apollo gives you a built-in explorer (**Sandbox**). Paste this query and run it:

```
the query
query {
  authors {
    name
    country
    books { title year }
  }
}
```

The server replies with exactly that shape — authors, each with their books nested inside, in one request:

```
the response
{ "data": {
    "authors": [
      { "name": "Asha Patel", "country": "India",
        "books": [
          { "title": "Databases 101", "year": 2020 },
          { "title": "SQL in Practice", "year": 2022 } ] },
      { "name": "Ravi Kumar", "country": "India",
        "books": [
```

```
          { "title": "Web Services Basics", "year": 2023 } ] }
    ] } }
```

Part 6 · Build the front-end (HTML + CSS)

Make a folder `public` inside your project. Create `public/style.css`:

public/style.css

```
* { box-sizing: border-box; }
body {
  font-family: system-ui, Arial, sans-serif;
  background: #f1f3f8;
  color: #1b1b2f;
  margin: 0;
  padding: 2rem;
}
h1 { color: #2e6f9e; margin: 0 0 0.3rem; }
.sub { color: #5d6577; margin: 0 0 1.5rem; }
.author-card {
  background: #fff;
  border: 1px solid #d6dae6;
  border-left: 6px solid #1b998b;
  border-radius: 10px;
  padding: 1rem 1.25rem;
  margin-bottom: 1rem;
  box-shadow: 0 2px 8px rgb(0 0 0 / 6%);
}
.author-card h2 { margin: 0 0 0.1rem; font-size: 1.1rem; }
.author-card .country { color: #5d6577; font-size: 0.85rem; margin-bottom: 0.5rem; }
.book { padding: 0.35rem 0.6rem; background: #f1f3f8; border-radius: 6px; margin: 0.3rem 0; font-size:
0.95rem; }
.book .yr { color: #2e6f9e; font-weight: 600; }
form {
  background: #fff; border: 1px solid #d6dae6; border-left: 6px solid #e8541e;
  border-radius: 10px; padding: 1rem 1.25rem; margin-top: 1.5rem;
}
form h3 { margin: 0 0 0.7rem; color: #e8541e; }
input, select { padding: 0.5rem; margin: 0.25rem 0.4rem 0.25rem 0; border: 1px solid #c9ccd6; border-radius:
6px; }
button { padding: 0.55rem 1rem; background: #2e6f9e; color: #fff; border: 0; border-radius: 6px; cursor:
pointer; font-weight: 600; }
```

Now `public/index.html`. The small `<script>` at the bottom does two things: it sends a GraphQL **query** to list authors, and the form sends a **mutation** to add a book. The `gql()` helper is just a `fetch()` POST — remember, a GraphQL call is a normal HTTP POST.

public/index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Campus Library</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Campus Library</h1>
  <p class="sub">Authors and their books &mdash; data served by GraphQL</p>

  <div id="list">Loading&hellip;</div>

  <form id="addForm">
    <h3>Add a book</h3>
    <input id="title" placeholder="Book title" required>
    <input id="year" type="number" placeholder="Year" style="width:90px">
    <select id="authorId">
      <option value="1">Asha Patel</option>
      <option value="2">Ravi Kumar</option>
    </select>
    <button type="submit">Add book</button>
  </form>

  <script>
```

```
    const API = 'http://localhost:4000/';

    // helper: send a GraphQL query/mutation to the API
    async function gql(query, variables) {
      const res = await fetch(API, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ query, variables }),
      });
      const json = await res.json();
      return json.data;
    }

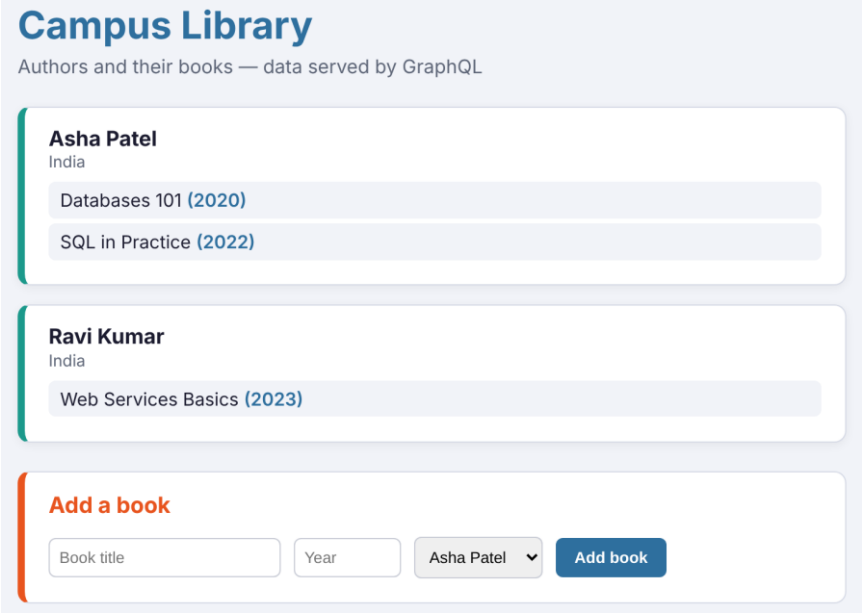
    // 1. READ: load all authors with their books
    async function load() {
      const data = await gql(`{ authors { id name country books { title year } } }`);
      document.getElementById('list').innerHTML = data.authors.map(a => `
        <div class="author-card">
          <h2>${a.name}</h2>
          <div class="country">${a.country || ''}</div>
          ${a.books.map(b => `<div class="book">${b.title} <span
class="yr">(${b.year})</span></div>`).join('')}
        </div>`).join('');
    }

    // 2. WRITE: add a book, then reload the list
    document.getElementById('addForm').addEventListener('submit', async (e) => {
      e.preventDefault();
      const title = document.getElementById('title').value;
      const year = Number(document.getElementById('year').value);
      const authorId = document.getElementById('authorId').value;
      await gql(
        `mutation($t:String!,$y:Int,$a:ID!){ addBook(title:$t, year:$y, authorId:$a){ id } }`,
        { t: title, y: year, a: authorId }
      );
      e.target.reset();
      load();
    });

    load();
  </script>
</body>
</html>
```

Part 7 · Run the whole app

Keep the API running (`node server.js`). Then, in VS Code, right-click `public/index.html` → **"Open with Live Server"**. Your browser opens the page (usually `http://127.0.0.1:5500`) and it immediately loads the data from GraphQL:

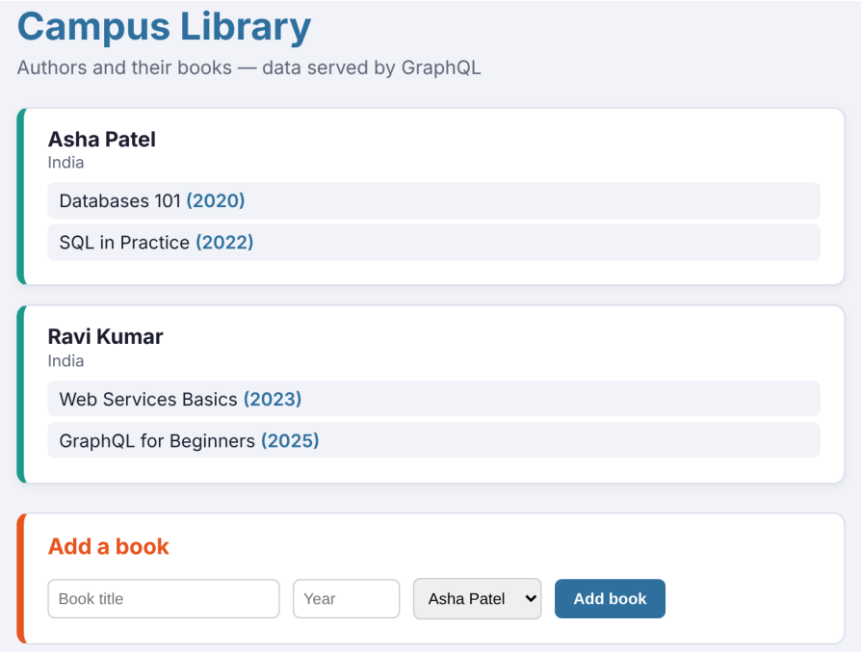


The live page — authors and their books, fetched from the GraphQL API in one request.

> **If the list says "Loading…" forever**
> The API isn't running or the port is wrong. Make sure `node server.js` is running and shows port 4000, and that `API` in index.html is `http://localhost:4000/`.

Part 8 · Add a book (the mutation)

In the **Add a book** form, type a title (e.g. `GraphQL for Beginners`), a year (`2025`), pick **Ravi Kumar**, and click **Add book**. The form sends a mutation, the server `INSERT`s into MySQL, and the list refreshes — the new book appears instantly:



After the mutation — "GraphQL for Beginners (2025)" now shows under Ravi Kumar, and it is saved in MySQL.

Because the data is really in MySQL now, you can confirm it from the database side too:

```
Terminal
mysql> SELECT title, year FROM books WHERE author_id = 2;
+-----------------------+------+
| title                 | year |
+-----------------------+------+
| Web Services Basics   | 2023 |
| GraphQL for Beginners | 2025 |
+-----------------------+------+
```

Recap:

- Stored data in **MySQL** (authors + books, linked by a foreign key).
- Put a **GraphQL API** in front of it with a schema and a few resolvers.
- Ran one **query** that fetched authors and their nested books in a single request.
- Ran one **mutation** from an HTML form that added a book and saved it to MySQL.

Your final project folder:

```
project structure
campus-library/
  schema.sql          <- the MySQL database + tables
  db.js               <- connection to MySQL
  server.js           <- the GraphQL schema + resolvers
  package.json
  public/
```

```
    index.html        <- the web page (query + mutation)
    style.css
```

Where to go next

Try adding a `deleteBook(id)` mutation, or a `books(year: Int)` filter argument. Each is one new line in the schema plus one small resolver.